

Kafka and Spring Kafka in Real Company Applications

Executive Summary

Preferred companies and industries were **unspecified**, so this research intentionally spans retail, marketplaces, mobility, SaaS, enterprise software, banking, and publishing/media. I biased the selection toward official engineering blogs that describe **business-facing event flows**, **application-level integrations**, and **code-bearing posts**, while deprioritizing posts that were primarily about broker operations, cluster tuning, or other infra-only concerns. The result is a curated set of production case studies plus implementation-oriented companion posts that together make a strong practical reading list.

1

Across the strongest production posts, Kafka is rarely used as “just a queue.” Companies used it as an **event backbone** for retail item setup and search indexing at Walmart, product-hierarchy aggregation and ranking at Zalando, internal message-routing at LINE, low-code ingress/egress and stream quality controls at Grab, content-publication joins at Michelin, and reliable third-party feed delivery at Jamf. The recurring pattern is: model domain changes as durable events; partition around ordering or tenancy needs; materialize domain views with stream processors; and add governance, retries, and observability where the business depends on correctness.

2

For Spring-specific practice, the companion posts and official Spring references converge on a common toolkit: `KafkaTemplate` for production, `@KafkaListener` with consumer groups and partition-aware concurrency for consumption, `@RetryableTopic` or `DefaultErrorHandler` plus dead-letter publishing for failures, schema-registry-backed Avro for message contracts, and `KafkaTransactionManager` / exactly-once support when a consume-process-produce flow must commit offsets and produced records atomically. Tests that include the framework and infrastructure instead of mocking everything out also show up repeatedly, using Spring Cloud Stream's test binder or Testcontainers-backed integration tests.

3

Curated Company Blog Posts

Production and business-focused posts

Title	Company	URL	Date	Summary emphasizing application context	Kafka and Spring features used	Code?
Apache Kafka for Item Setup	Walmart Global Tech	medium.com/walmartglobaltech/apache-kafka-for-item-setup-3fe8f4ba5967	Oct 16, 2016 	Walmart's "Item Setup" flow merges product, offer, price, inventory, and logistics data from many heterogeneous sources. Modeling changes as Kafka event streams helped the company onboard sellers faster, update listings faster, and power a near-real-time search index where site changes appear in seconds. 	Topics, event streams as system-of-record input, many service topologies, schema-management challenges, multi-DC balancing, near-real-time downstream indexing. 	No

Title	Company	URL	Date	Summary emphasizing application context	Kafka and Spring features used	Code?
Building Walmart's Seamless Communication: Leveraging Kafka's Custom Partitioning	Walmart Global Tech	medium.com/walmartglobaltech/building-walmarts-seamless-communication-leveraging-kafka-s-custom-partitioning-5d75a2ea61a0	Mar 13, 2024 ⁵	This post ties Kafka partitioning directly to customer communication outcomes such as timely grocery-delivery notifications and SLA-sensitive messaging. Walmart shows how custom partitioning can support multi-tenancy, prioritization, throttling, and geography-aware routing without exploding topic counts. ⁵	Topics, partitions, custom <code>Partitioner</code> , tenant isolation, SLA prioritization, throttling, geography-aware routing. ⁵	Yes

Title	Company	URL	Date	Summary emphasizing application context	Kafka and Spring features used	Code?
Event First Development - Moving Towards Kafka Pipeline Applications	Zalando	engineering.zalando.com/posts/2017/10/event-first-development---moving-towards-kafka-pipeline-applications.html	Oct 10, 2017 ⁶	Zalando describes redesigning core article-data services in its Fashion Platform so that data is easier to access across teams and future use cases. The post is important because it frames Kafka not as transport but as the center of an event-first product-data platform. ⁶	Event-driven architecture, pipeline applications, domain-event orientation, data democratization. ⁶	No

Title	Company	URL	Date	Summary emphasizing application context	Kafka and Spring features used	Code?
Many-to-Many Relationships Using Kafka	Zalando	engineering.zalando.com/posts/2018/05/many-to-many-using-kafka.html	May 8, 2018 ⁷	In Zalando's Smart Product Platform, Kafka is used to sequence events and build aggregated views of product hierarchies. The post focuses on the hard part of event-driven catalog systems: real-time joins and many-to-many entity relationships without giving up domain clarity. ⁷	Topics, sequencing, aggregation, real-time joins, partition broadcasting for shared reference data. ⁸	Yes

Title	Company	URL	Date	Summary emphasizing application context	Kafka and Spring features used	Code?
Real-time Ranking with Apache Kafka's Streams API	Zalando	engineering.zalando.com/posts/2017/11/real-time-ranking-kafka.html	Nov 23, 2017 ⁹	Zalando used Kafka Streams to build distributed Scala components for real-time domain ranking in AWS. The business value is explicit: ranking is treated as an application on top of Kafka, not merely analytics beside it. ¹⁰	Kafka Streams, containers, distributed scaling, application-level stream processing. ¹¹	Minimal
Applying Kafka Streams for internal message delivery pipeline	LINE	engineering.linecorp.com/en/blog/applying-kafka-streams-for-internal-message-delivery-pipeline/	Aug 18, 2016 ¹²	LINE used Kafka Streams in a core internal message-delivery pipeline, explicitly noting that the application affects service performance and reliability. The post shows filtering and categorizing logs/events into derived topics for downstream internal use. ¹³	Kafka Streams, application IDs, rebalancing through Kafka coordination, KStream transformation/ filtering into derived topics. ¹⁴	Yes

Title	Company	URL	Date	Summary emphasizing application context	Kafka and Spring features used	Code?
How Kafka Connect helps move data seamlessly	Grab	engineering.grab.com/kafka-connect	Apr 6, 2022 ¹⁵	Grab's Coban platform added Kafka Connect to make moving data into and out of Kafka a low-code experience for many internal teams. The post is useful because it connects Kafka platform work directly to transactional and analytical production use cases rather than presenting Connect as isolated plumbing. ¹⁶	Kafka Connect, schema-registry-backed conversion, Kubernetes deployment model, topics/clusters/connectors as managed resources. ¹⁷	Minimal

Title	Company	URL	Date	Summary emphasizing application context	Kafka and Spring features used	Code?
Real-time data quality monitoring: Kafka stream contracts with syntactic and semantic test	Grab	engineering.grab.com/real-time-data-quality-monitoring	Nov 26, 2025 ¹⁸	Grab turned Kafka data quality into an application concern by letting stream owners define data contracts, semantic rules, and observability metadata, then executing those checks in real time. The system publishes quality-issue events, stores bad records for analysis, and alerts owners, which makes it a strong example of event-driven governance. ¹⁹	Stream schema checks, semantic rules, dedicated consumer groups, issue events on Kafka, sink to S3, Slack alerting, UI observability. ¹⁹	Minimal

Title	Company	URL	Date	Summary emphasizing application context	Kafka and Spring features used	Code?
Retryable Topics with Spring Kafka	Jamf	medium.com/jamf-engineering/retryable-topics-with-spring-kafka-946360f2d644	Jun 7, 2022	Jamf's practical example is grounded in customer-facing integration feeds for network traffic, threats, applications, and devices. Messages were enriched and pushed to Syslog, HTTP, or S3 endpoints, and the company designed a Spring Kafka retry-queue pattern because downstream systems sometimes go unavailable and dropping data was unacceptable.	Spring Kafka, <code>@RetryableTopic</code> , retry topics, DLT handling, custom backoff, listener/container factories, <code>KafkaTemplate</code> for recovery publication.	Yes

Title	Company	URL	Date	Summary emphasizing application context	Kafka and Spring features used	Code?
The Michelin Guide: an unexpected event driven use case	Michelin	blogit.michelin.io/the-michelin-guide-an-unexpected-event-driven-use-case/	Dec 12, 2022 ²³	Michelin used Kafka to reduce Michelin Guide publication delays from roughly two days to near real time by combining change streams from editorial/content systems and Salesforce CDC. A Kafka Streams application joins translations, pictures, and restaurant state, filters publication rules, and routes content to interim and production environments, including special handling for star-revelation events. ²⁴	Kafka Connect for CDC, Kafka Streams, KTable/KStream joins, state stores, keyed aggregation, environment-specific output topics. ²⁵	Yes

Spring and implementation-oriented companion posts

Title	Company	URL	Date	Summary emphasizing application context	Kafka and Spring used
Streaming with Spring Cloud	Walmart Global Tech	medium.com/walmartglobaltech/streaming-with-spring-cloud-24a001ad307a	Jul 24, 2020 ²⁶	This is an implementation post rather than a single production case study, but it is a strong companion because it shows how Walmart engineers think about asynchronous workflows, eventual consistency, scaling, retries, and framework-level testing. It is especially useful for teams building event-driven microservices with Spring idioms rather than raw Kafka clients. ²⁷	Spring Cloud Stream, consumer group, concurrent consumers, retries/backoff, transactional model, test bindings
Working with Testcontainers	Backbase	engineering.backbase.com/2024/05/06/working-with-testcontainers/	May 6, 2024 ²⁸	Backbase uses a product-price-changed business event to show how to integration-test Spring services against a real Kafka container and database. The point is not broker administration; it is verifying asynchronous application behavior deterministically, including database state changes after Kafka-driven processing. ²⁹	Spring Boot testcontainers, Testcontainers Kafka, bootstrap-server, asynchronous verification, event-emitter pattern
Integrating Spring Boot with Red Hat Integration Service Registry	Red Hat	developers.redhat.com/blog/2021/02/15/integrating-spring-boot-with-red-hat-integration-service-registry	Feb 15, 2021 ³¹	This post is implementation-oriented, but it distills a very common enterprise use case from field work: event-driven microservices on Kafka with centralized schema governance. It shows how Spring Boot producer and consumer clients use Avro plus a registry so teams can validate compatibility and evolve contracts safely. ³¹	Spring Kafka, KafkaListener, @KafkaListener, acknowledgments, schema registry, rules, serializers ³²

Title	Company	URL	Date	Summary emphasizing application context	Kafka and Spring used
Building a Kafka Client: Enhancing Throughput with Consumer Groups and Multiple Clients	Vendavo	engineering.vendavo.com/building-a-kafka-client-enhancing-throughput-with-consumer-groups-and-multiple-clients-a976e92a5220	Oct 18, 2023 ³³	Vendavo frames the problem through Spring-based applications that need better throughput on real-time commercial data streams. The post is especially useful for practicing consumer-group mechanics and when to add multiple consumers per topic. ³⁴	Spring Kafka, <code>@KafkaListener</code> consumer group throughput scaling configuration in application. ³⁵
Implementing a basic Kafka Producer and Consumer using Spring Boot, Spring Kafka and Avro schema	ING Hubs Romania	medium.com/ing-tech-romania/implementing-a-basic-kafka-producer-and-consumer-using-spring-boot-spring-kafka-and-avro-schema-2b6d06e6c4cf	May 16, 2024 ³⁶	ING's post is a concise implementation companion for teams who want a typed event model rather than loose JSON. It demonstrates Avro-based event classes, schema-registry wiring, producer/consumer properties, and <code>ErrorHandlingDeserializer</code> for robustness. ³⁷	Spring Kafka, Kafka producer/consumer, Avro, schema registry, <code>ErrorHandlingDeserializer</code> , <code>acks=all</code> , retries
Shooting ourselves in the foot by enabling Spring Kafka batching	One2N	one2n.io/blog/shooting-ourselves-in-the-foot-by-enabling-spring-kafka-batching	Mar 16, 2026 ³⁹	One2N documents a real performance-tuning mistake in a high-volume Spring Kafka workload that processed about 14 million messages per day. The post is valuable because it explains when batch listeners help, when they do not, and why losing non-blocking retries can be a bad trade for record-oriented work. ⁴⁰	Spring Kafka batch non-blocking retries incompatibility, <code>batching (batchListener.linger.ms)</code> , <code>max.poll.records</code> , <code>RecordBatch</code> serializer

Implementation Patterns and Spring Kafka Snippets

A clear set of implementation patterns emerges when the case-study posts are read alongside official Spring Kafka documentation. The strongest teams start with **domain events**, then use partitions and consumer groups to scale safely, add stream joins or materialized views when the domain needs enrichment, and treat retries, contracts, and observability as first-class parts of the application rather than afterthoughts. ⁴²

Pattern	What the posts show	Typical Spring and Kafka ingredients
Event-first domain backbone	Walmart, Zalando, and Michelin all model business changes as event streams first, then derive listings, search indexes, aggregated catalog views, or publishable restaurant records from those streams. ⁴³	Domain topics, stable event keys, downstream materialized views, explicit event versioning.
Partition-aware scaling	Walmart and Vendavo both connect throughput and SLA behavior to partitioning and consumer-group design. One2N adds the caution that not every workload benefits from batch consumption just because throughput matters. ⁴⁴	Good partition key selection, consumer groups, listener concurrency, selective batching only when downstream effects can be batched.
Stream enrichment and join pipelines	Zalando, LINE, and Michelin all use stream processors to join, filter, categorize, or rank event flows close to the business logic itself. ⁴⁵	Kafka Streams, KTable/KStream joins, state stores, derived topics, application IDs.
Retry isolation instead of “poison-message blocking”	Jamf’s dedicated retry topics and Spring’s retry infrastructure show the mainstream pattern for transient downstream failure. One2N and Spring’s docs show the anti-pattern: mixing batch listeners with non-blocking retry expectations. ⁴⁶	<code>@RetryableTopic</code> , DLTs, backoff, DLT handlers, <code>DefaultErrorHandler</code> , <code>DeadLetterPublishingRecoverer</code> .
Schema governance as an application concern	Red Hat, ING, and Grab treat schemas/contracts as runtime behavior, not documentation only. Grab extends this further into semantic rules and issue events. ⁴⁷	Avro, schema registry, compatibility checks, <code>ErrorHandlingDeserializer</code> , contract metadata, issue topics.
Realistic async testing and observability	Walmart’s Spring Cloud Stream post, Backbase, and Grab all argue for system-level verification: test binders or Kafka containers for tests, plus alerts and visibility into bad records in production. ⁴⁸	Spring Cloud Stream test binder, Testcontainers, Awaitility-style assertions, dedicated observability topics and alerts.

The code below is **synthesized** from the recurring patterns in the company posts and official Spring Kafka references. It is not copied verbatim from a single source, but it preserves the implementation shape they converge on. ⁴⁹

A common producer shape is a single `KafkaTemplate` backed by Spring Boot Kafka properties. Red Hat and ING both use that pattern, and Spring's own reference shows `KafkaTemplate` as the central producer abstraction. ⁵⁰

```
@Configuration
class ProducerConfiguration {

    @Bean
    ProducerFactory<String, Message> producerFactory(KafkaProperties
kafkaProperties) {
        return new
DefaultKafkaProducerFactory<>(kafkaProperties.buildProducerProperties());
    }

    @Bean
    KafkaTemplate<String, Message> kafkaTemplate(ProducerFactory<String,
Message> pf) {
        return new KafkaTemplate<>(pf);
    }
}

@Service
@RequiredArgsConstructor
class MessagePublisher {
    private final KafkaTemplate<String, Message> kafkaTemplate;

    public CompletableFuture<SendResult<String, Message>> publish(
        String topic, String key, Message event) {
        return kafkaTemplate.send(topic, key, event);
    }
}
```

For consumers, the recurring production pattern is still annotation-driven and intentionally boring: a topic, a group, headers or metadata when needed, and manual acknowledgment when the application wants explicit control over offset progression. That is exactly the posture shown in the Red Hat article and the Spring Kafka listener docs. ⁵¹

```
@EnableKafka
@Configuration
class ConsumerConfiguration { }
```

```

@Component
class MessageListener {

    @KafkaListener(topics = "messages", groupId = "message-consumers")
    public void handle(
        @Payload Message message,
        @Header(KafkaHeaders.RECEIVED_TOPIC) String topic,
        @Header(KafkaHeaders.OFFSET) long offset,
        Acknowledgment acknowledgment) {

        process(message);
        acknowledgment.acknowledge();
    }

    private void process(Message message) {
        // business logic
    }
}

```

For transient failures, Jamf's production story is the clearest example: failed deliveries become recovery messages, are retried on dedicated retry topics with increasing delay, and ultimately land in a DLT path if attempts are exhausted. Spring's recommended default is `@RetryableTopic`; for batch listeners, the docs explicitly say to use `DefaultErrorHandler` plus `DeadLetterPublishingRecoverer` instead. One2N's incident report is the practical reminder that non-blocking retries and batch listeners do not compose the way many teams first assume. ⁵²

```

@Component
class ReliableIntegrationListener {

    @RetryableTopic(
        attempts = "4",
        backoff = @Backoff(delay = 2_000),
        dltTopicSuffix = ".dlt"
    )
    @KafkaListener(topics = "integration.recovery", groupId = "delivery-
workers")
    public void redeliver(RecoveryMessage msg) {
        sendToExternalTarget(msg); // HTTP / Syslog / S3 / partner API
    }

    @DltHandler
    public void deadLetter(RecoveryMessage msg) {
        persistFailure(msg); // audit table, ops alert, replay queue,
etc.
    }
}

```

```

    }
}

```

For transactions and exactly-once, the official Spring references are the main primary source. The practical rule is straightforward: if a transaction is active, `KafkaTemplate` sends participate in it; if the listener container is configured with a Kafka-aware transaction manager, consumed offsets are sent to the same transaction before commit. That is the Spring abstraction most teams should practice before attempting more ambitious cross-system choreography. ⁵³

```

@Service
@RequiredArgsConstructor
class LedgerService {
    private final KafkaTemplate<String, LedgerEvent> kafkaTemplate;
    private final LedgerRepository ledgerRepository;

    @Transactional
    public void processAndEmit(List<LedgerEvent> events) {
        events.forEach(e -> kafkaTemplate.send("ledger-events", e.accountId(),
e));
        ledgerRepository.saveAll(events.stream().map(this::toEntity).toList());
    }

    public boolean publishAtomically(LedgerEvent first, LedgerEvent second) {
        return kafkaTemplate.executeInTransaction(ops -> {
            ops.send("ledger-events", first.accountId(), first);
            ops.send("ledger-events", second.accountId(), second);
            return true;
        });
    }
}

```

For message schemas, Red Hat and ING show the same core pattern with different registries: Avro serializers/deserializers, registry endpoint properties, and consumer-side handling for bad payloads. Grab's contract-monitoring post is a useful reminder that **structural compatibility is necessary but not sufficient**; semantic rules matter too. ⁴⁷

```

# producer
spring.kafka.bootstrap-servers=${kafka.bootstrap-servers}
spring.kafka.producer.key-
serializer=org.apache.kafka.common.serialization.StringSerializer
spring.kafka.producer.value-
serializer=io.apicurio.registry.utils.serde.AvroKafkaSerializer
spring.kafka.producer.properties.apicurio.registry.url=${registry.url}

# consumer

```



```

spring.kafka.listener.ack-mode=manual
spring.kafka.consumer.group-id=message-consumers
spring.kafka.consumer.auto-offset-reset=earliest
spring.kafka.consumer.enable-auto-commit=false
spring.kafka.consumer.key-
deserializer=org.apache.kafka.common.serialization.StringDeserializer
spring.kafka.consumer.value-
deserializer=io.apicurio.registry.utils.serde.AvroKafkaDeserializer
spring.kafka.consumer.properties.apicurio.registry.url=${registry.url}
spring.kafka.consumer.properties.apicurio.registry.use-specific-avro-reader=true

```

Finally, the testing pattern is unusually consistent: do not rely only on mocks for asynchronous flows. Walmart argues for deterministic framework-level tests with the Spring Cloud Stream test binder, and Backbase shows the Testcontainers equivalent with a real Kafka broker and database in the loop. ⁵⁴

```

@SpringBootTest
@Testcontainers
class ProductPriceChangedEventHandlerTest {

    @Container
    static final KafkaContainer kafka =
        new KafkaContainer(DockerImageName.parse("confluentinc/cp-kafka:
7.5.1"));

    @DynamicPropertySource
    static void overrideProperties(DynamicPropertyRegistry registry) {
        registry.add("spring.kafka.bootstrap-servers",
kafka::getBootstrapServers);
    }

    // emit event, wait for async processing, assert database state
}

```

Practice Scenarios

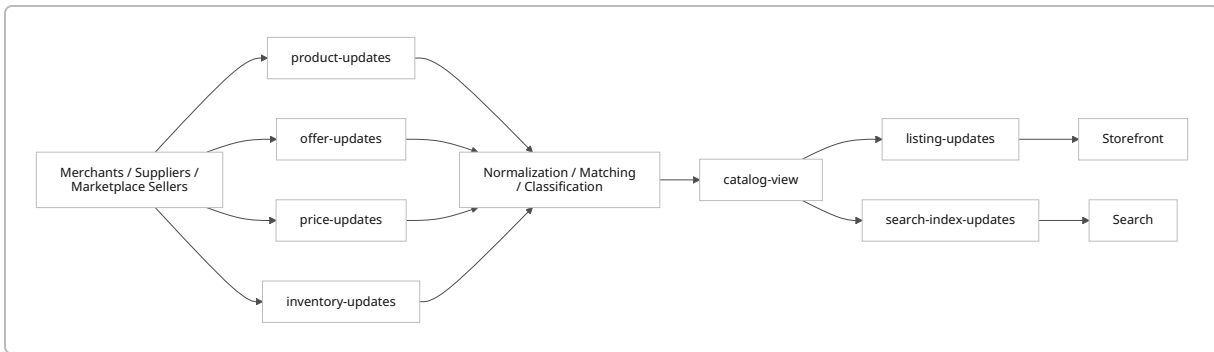
The table below turns the collected posts into **hands-on practice labs**. The details are intentionally concrete enough to implement in Spring Boot plus Kafka, but still generic enough to adapt to your stack.

Scenario	Inspired by	Goal	Key events and topics	Suggested schema	Features
Retail item onboarding and search indexing	Walmart Item Setup ⁴	Ingest seller and catalog changes, normalize them, and publish a search-ready product view quickly.	product-updates, offer-updates, price-updates, inventory-updates, search-index-updates	ProductChanged, OfferChanged, PriceAdjusted, InventoryAdjusted, SearchDocumentReady	Keyed SKU/product materialized indexes
Customer notification prioritization and tenant throttling	Walmart custom partitioning ⁵	Deliver time-sensitive notifications while isolating premium tenants and avoiding downstream overload.	customer-communications, customer-communications-priority, delivery-audit	NotificationRequested with tenantId, priority, region, channel, deadline	Custom SLA based throttling
Smart catalog hierarchy aggregation	Zalando Event First + Many-to-Many ⁵⁵	Build a local materialized view that resolves article/product/attribute relationships in real time.	articles, products, attributes, catalog-view	ArticleChanged, ProductChanged, AttributeChanged, CatalogViewUpdated	Event-driven many-to-many data feeds
Real-time domain or product ranking	Zalando real-time ranking ⁵⁶	Compute domain or product scores continuously instead of waiting for batch ranking.	click-events, link-events, rank-input, rank-output	LinkObserved, RankScoreUpdated	Kafka distributed

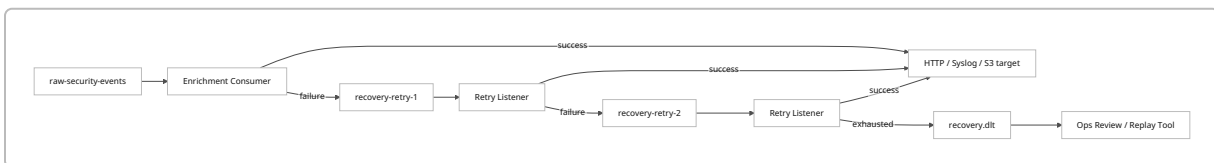
Scenario	Inspired by	Goal	Key events and topics	Suggested schema	Features
Internal message categorization pipeline	LINE ¹⁴	Consume an internal operations/ log topic and split it into derived topics by event type or category.	operation-log , operation-log-by-type-*	OperationLog with opType , service , ts , payload	Kafka derived
CDC-driven content publication	Michelin Guide ²⁵	Join CDC streams from multiple SaaS systems to publish only valid, translated, image-ready entities.	accounts_changes , contenthub-assets , restaurants , restaurants-active , restaurants-release	RestaurantChangeEvent , AssetChanged , RestaurantPublished	Kafka KTable stores topics
Managed ingress/ egress data hub	Grab Kafka Connect ¹⁷	Let internal teams move data into and out of Kafka with minimal custom code.	source-*, sink-*, connector-status	Source-specific messages plus connector metadata/ status records	Kafka conve mana

Scenario	Inspired by	Goal	Key events and topics	Suggested schema	Features
Stream data quality contract enforcement	Grab data quality monitoring ¹⁹	Validate structural and semantic rules on live topics and emit issue events when data goes bad.	payments, orders, quality-issues, quality-observability	DataContract, DataQualityIssue, plus topic-specific events	Schemas, dedicated issue
Reliable third-party feed delivery	Jamf ²¹	Enrich events, then forward them to a flaky external system without dropping records.	raw-security-events, enriched-events, recovery, recovery-retry-*, recovery.dlt	RecoveryMessage with target identity, payload, attempt metadata	@Retention, backoff, replay
Schema-governed financial event exchange	Red Hat + ING ⁵⁷	Publish typed transaction events with registry-backed schemas and safe consumer handling.	transaction_events_topic, transactions.dlt	Avro TransactionEvent with transaction id, amount, currency, status, timestamp	Avro, Kafka, @Kafka, Error, manual

Two sample flows are especially worth practicing because they combine business value with implementation richness. The first is a retail item-to-index pipeline inspired by Walmart's item-setup architecture. ⁴



The second is a Spring Kafka retry topology inspired by Jamf’s feed-delivery design and reinforced by Spring’s retry-topic guidance. It is one of the best hands-on labs because the happy path is easy, but the failure modes teach offset handling, backoff, DLTs, and replay discipline. ⁵⁸



Common Patterns and Anti-Patterns

Theme	Pattern that appears repeatedly	Anti-pattern that appears repeatedly	Evidence
Domain modeling	Successful teams publish domain changes and derive downstream views from them.	Treating Kafka as a dumb pipe encourages hidden coupling and weak reuse.	Walmart item setup, Zalando’s event-first platform, and Michelin’s content pipeline all emphasize event modeling over ad hoc integration. ⁵⁹
Partition strategy	Keys, partitions, and consumer groups are chosen to protect ordering and SLA behavior.	Default hashing or arbitrary batching is not enough when priorities, tenants, or geographic load matter.	Walmart custom partitioning and Vendavo’s consumer-group guidance show the positive pattern; One2N shows what happens when batching assumptions are misapplied. ⁴⁴
Stream enrichment	Use Kafka Streams or similar runtime processors when the business logic is fundamentally continuous.	Falling back to batch or orchestration-heavy designs can reintroduce latency and coordination pain.	Zalando, LINE, and Michelin all keep joins and transformations in stream applications close to the event flow. ⁴⁵

Theme	Pattern that appears repeatedly	Anti-pattern that appears repeatedly	Evidence
Failure handling	Isolate transient failures with retry topics, backoff, DLTs, and explicit replay paths.	Dropping messages or letting one poison message block an entire flow is repeatedly treated as unacceptable.	Jamf's recovery topics and Spring's retry/DLT references are the clearest positive examples. ⁶⁰
Contracts and schemas	Typed events, registries, and data contracts reduce breakage and make reuse possible across teams.	"JSON plus tribal knowledge" scales poorly, especially with many attributes, many teams, or semantic assumptions.	Walmart notes schema-management pain; Red Hat and ING show schema-registry solutions; Grab extends contracts to semantic checks. ⁶¹
Testing and observability	The better posts test against real async infrastructure and add alerting/diagnostic surfaces.	<code>Thread.sleep</code> -style waiting and poor poison-data visibility make failures harder to reproduce and fix.	Walmart's test-binder discussion, Backbase's Testcontainers example, and Grab's quality issue events/UI are consistent on this. ⁴⁸
Spring Kafka specifics	Record listeners with clear retry semantics are the safest default for many application workloads.	Enabling batch listeners "for performance" without proving that downstream work actually batches is a classic foot-gun.	Spring docs state non-blocking retries are not supported with batch listeners, and One2N's incident is the exact real-world version of that warning. ⁶²

Open Questions and Limitations

This report intentionally favored **official company engineering blogs** and **English-language posts**. A few excellent Kafka engineering posts were excluded because they were primarily about infrastructure operations rather than business application flows, even when technically strong.

Some official pages, especially Medium-backed publications and blog indexes, expose enough public text to verify the title, date, architecture, and presence of code examples, but not always every full code block. Where that happened, the "Code?" assessment is marked **Minimal** rather than overstating certainty.

A final practical limitation is topical balance: there are many more strong public posts about **Kafka Streams, partitions, and event modeling** than there are public, detailed writeups about **Spring Kafka transactions and exactly-once in production**. For those two areas, the most reliable primary references remain the official Spring Kafka documentation, which I used as the authoritative companion source for the implementation section. ⁵³

- 1 2 4 42 43 59 61 <https://medium.com/walmartglobaltech/apache-kafka-for-item-setup-3fe8f4ba5967>
<https://medium.com/walmartglobaltech/apache-kafka-for-item-setup-3fe8f4ba5967>
- 3 <https://docs.spring.io/spring-kafka/reference/kafka/sending-messages.html>
<https://docs.spring.io/spring-kafka/reference/kafka/sending-messages.html>
- 5 44 <https://medium.com/walmartglobaltech/building-walmarts-seamless-communication-leveraging-kafka-s-custom-partitioning-5d75a2ea61a0>
<https://medium.com/walmartglobaltech/building-walmarts-seamless-communication-leveraging-kafka-s-custom-partitioning-5d75a2ea61a0>
- 6 55 <https://engineering.zalando.com/posts/2017/10/event-first-development---moving-towards-kafka-pipeline-applications.html>
<https://engineering.zalando.com/posts/2017/10/event-first-development---moving-towards-kafka-pipeline-applications.html>
- 7 8 <https://engineering.zalando.com/posts/2018/05/many-to-many-using-kafka.html>
<https://engineering.zalando.com/posts/2018/05/many-to-many-using-kafka.html>
- 9 Real-time Ranking with Apache Kafka's Streams API
https://engineering.zalando.com/posts/2017/11/real-time-ranking-kafka.html?utm_source=chatgpt.com
- 10 11 45 56 <https://engineering.zalando.com/posts/2017/11/real-time-ranking-kafka.html>
<https://engineering.zalando.com/posts/2017/11/real-time-ranking-kafka.html>
- 12 13 14 <https://engineering.linecorp.com/en/blog/applying-kafka-streams-for-internal-message-delivery-pipeline/>
<https://engineering.linecorp.com/en/blog/applying-kafka-streams-for-internal-message-delivery-pipeline/>
- 15 How Kafka Connect helps move data seamlessly
https://engineering.grab.com/kafka-connect?utm_source=chatgpt.com
- 16 17 <https://engineering.grab.com/kafka-connect>
<https://engineering.grab.com/kafka-connect>
- 18 19 <https://engineering.grab.com/real-time-data-quality-monitoring>
<https://engineering.grab.com/real-time-data-quality-monitoring>
- 20 21 22 46 52 58 60 <https://medium.com/jamf-engineering/retryable-topics-with-spring-kafka-946360f2d644>
<https://medium.com/jamf-engineering/retryable-topics-with-spring-kafka-946360f2d644>
- 23 24 25 <https://blogit.michelin.io/the-michelin-guide-an-unexpected-event-driven-use-case/>
<https://blogit.michelin.io/the-michelin-guide-an-unexpected-event-driven-use-case/>
- 26 27 48 54 <https://medium.com/walmartglobaltech/streaming-with-spring-cloud-24a001ad307a>
<https://medium.com/walmartglobaltech/streaming-with-spring-cloud-24a001ad307a>
- 28 Working with Testcontainers
https://engineering.backbase.com/2024/05/06/working-with-testcontainers/?utm_source=chatgpt.com
- 29 30 <https://engineering.backbase.com/2024/05/06/working-with-testcontainers/>
<https://engineering.backbase.com/2024/05/06/working-with-testcontainers/>

31 32 47 49 50 51 57 <https://developers.redhat.com/blog/2021/02/15/integrating-spring-boot-with-red-hat-integration-service-registry>

<https://developers.redhat.com/blog/2021/02/15/integrating-spring-boot-with-red-hat-integration-service-registry>

33 34 35 <https://engineering.vendavo.com/building-a-kafka-client-enhancing-throughput-with-consumer-groups-and-multiple-clients-a976e92a5220>

<https://engineering.vendavo.com/building-a-kafka-client-enhancing-throughput-with-consumer-groups-and-multiple-clients-a976e92a5220>

36 37 38 <https://medium.com/ing-tech-romania/implementing-a-basic-kafka-producer-and-consumer-using-spring-boot-spring-kafka-and-avro-schema-2b6d06e6c4cf>

<https://medium.com/ing-tech-romania/implementing-a-basic-kafka-producer-and-consumer-using-spring-boot-spring-kafka-and-avro-schema-2b6d06e6c4cf>

39 40 41 <https://one2n.io/blog/shooting-ourselves-in-the-foot-by-enabling-spring-kafka-batching>

<https://one2n.io/blog/shooting-ourselves-in-the-foot-by-enabling-spring-kafka-batching>

53 <https://docs.spring.io/spring-kafka/reference/kafka/transactions.html>

<https://docs.spring.io/spring-kafka/reference/kafka/transactions.html>

62 <https://docs.spring.io/spring-kafka/reference/kafka/annotation-error-handling.html>

<https://docs.spring.io/spring-kafka/reference/kafka/annotation-error-handling.html>